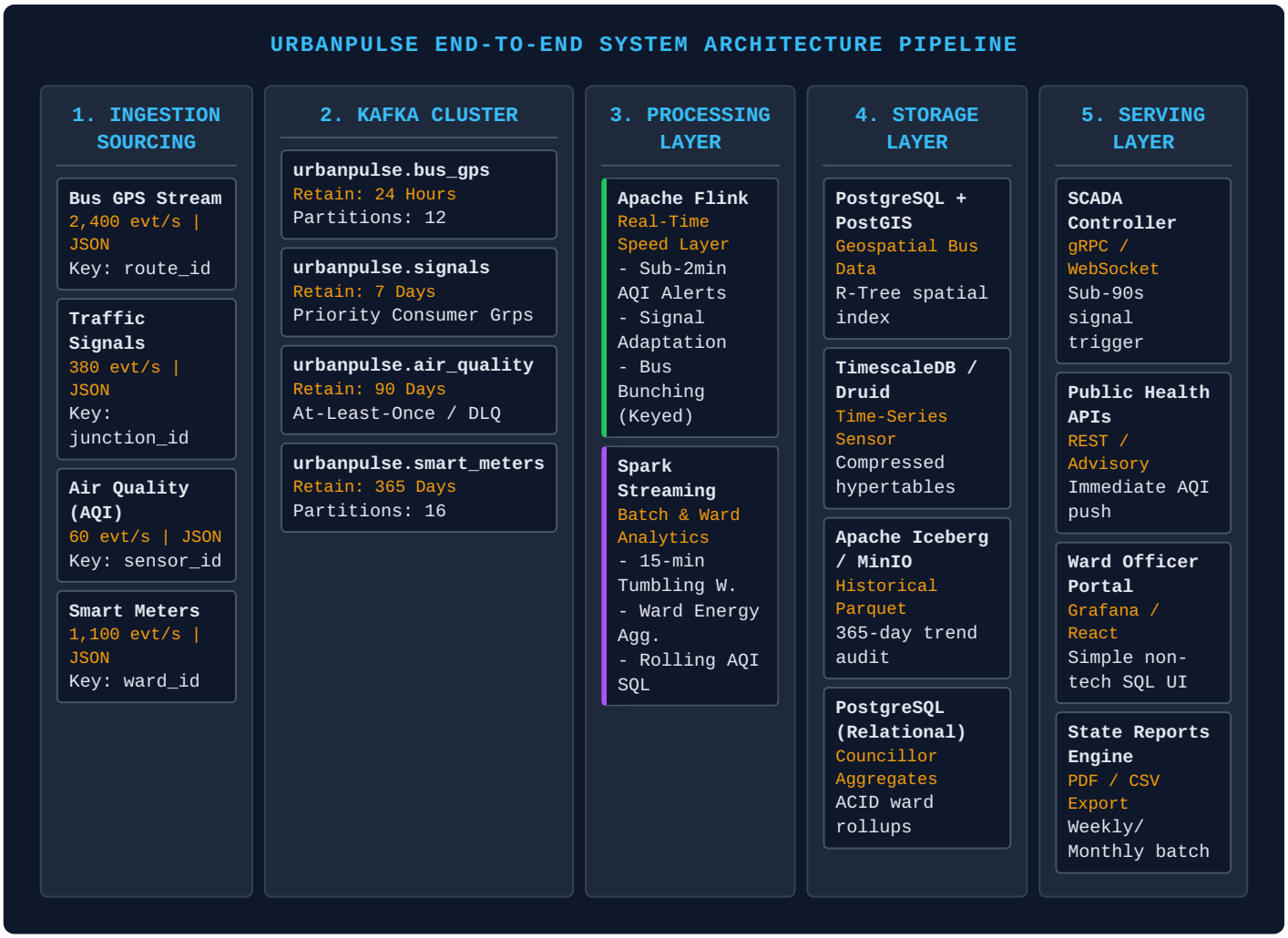
UrbanPulse: Architecture Design & Specification

Task A Deliverable — Lambda vs Kappa Evaluation & Government Architecture Readiness

Course Code	Target City	Total Marks Weight	Lead Architect
DSE ZG556 / CC ZG556	MetroConnect (Pop. 4.2M)	20 Marks (Task A)	Streaming Solutions Architect

1. UrbanPulse System Architecture & Layer Specifications

UrbanPulse is MetroConnect's unified real-time urban operations platform designed to ingest and analyze high-velocity streams from 12,000 GPS-equipped buses (~2,400 events/sec), 3,800 traffic signal controllers (~380 events/sec), 600 air quality sensors (~60 events/sec), and 1.1 million smart electricity meters (~1,100 events/sec). The system bridges sub-90-second operational response with long-term councilor batch reporting.



Database Technology Selection & Detailed Justification

To fulfill both real-time operational response and deterministic historical reporting, the storage layer utilizes purpose-built storage engines suited to each data modality:

Data Modality	Chosen Technology	Technical Rationale & Architectural Justification
Geospatial Bus Positions (`urbanpulse.bus_gps`)	**PostgreSQL with PostGIS Extension**	Supports high-throughput spatial indexing (GiST / R-Tree indexes). Enables efficient ST_DWithin geospatial distance calculations required for real-time bus bunching detection (<200 meters proximity) and map matching against route shape geometries.
Time-Series Sensor Data (AQI Sensors & Signals)	**TimescaleDB** (or Apache Druid)	Built on PostgreSQL, providing automatic partitioning over time and space (Hypertables). Offers high-rate concurrent ingestion (>10k writes/sec), 90%+ native columnar compression ratios, and continuous aggregate views for instant time-bucketed queries.
Historical Records & Audits (Smart Meters & Long-term AQI)	**Apache Iceberg on MinIO** (S3-compatible Object Store)	Columnar Parquet storage format hosted on-premises. Apache Iceberg provides ACID transactions, schema evolution, and time-travel querying across multi-terabyte historical datasets required for state energy audits (365 days) and multi-year pollution trends.
Councillor Report Aggregates (Ward Summaries)	**PostgreSQL Relational OLAP Cache**	Stores pre-computed 15-minute, daily, and monthly ward aggregations. Provides full ANSI SQL compliance and strict ACID guarantees required for regulatory submissions and low-latency dashboard queries accessed by non-technical ward officers.

2. Lambda vs Kappa Architecture Evaluation Matrix

MetroConnect requires both sub-2-minute emergency interventions and immutable, reproducible monthly government reports. Below is the formal evaluation comparing Lambda and Kappa architectures for UrbanPulse.

Evaluation Dimension	Lambda Architecture Assessment	Kappa Architecture Assessment
Latency	**Optimal for hybrid needs:** Speed layer (Flink) processes raw streams in sub-second memory pipelines for instant signal adaptation (90s) and AQI alerts (<2 min). Batch layer (Spark) produces deterministic 15-min ward rollups asynchronously without slowing down speed pipelines.	**Sub-second end-to-end:** Single stream engine handles all computations. While real-time alerts execute with sub-second latencies, heavy historical windowing aggregations can introduce backpressure into the single streaming pipeline if unisolated.
Fault Tolerance & Consistency	**Dual-guarantee resilience:** Speed layer trade-offs (e.g., eventual consistency in memory) are rectified every 15–60 minutes when the immutable batch layer recomputes truth from raw Parquet data stored on MinIO, guaranteeing 100% deterministic report accuracy.	**Checkpoint / Savepoint dependent:** Relies on Flink/ Spark state snapshotting and Kafka log retention. If state corruption occurs, recomputing state requires rewinding Kafka offsets, which can lead to transient calculation drift during active replaying.
Operational Complexity	**High maintenance overhead:** Requires maintaining two distinct codebases (e.g., Flink Java API for speed + Spark Scala/SQL for batch), separate cluster infrastructure, and dual storage sync logic. Increases engineering overhead for city IT staff.	**Low/Unified maintenance:** Single stream engine codebase (e.g., Flink SQL or Spark Structured Streaming) simplifies deployment, CI/CD, and monitoring. Only one processing framework needs to be managed by city operators.
Reprocessing Capability	**Simple & Deterministic:** Reprocessing past data (e.g., re-running 90 days of AQI data under a new government formula) is executed seamlessly as an offline batch job over Parquet data without impacting real-time incident processing.	**High Kafka Retention required:** Reprocessing requires replaying raw events from Kafka topics. Storing 365 days of high-velocity smart meter data (~1,100 msg/sec = ~35B events/yr) inside Kafka storage drives extremely high disk storage costs on Kafka brokers.

Evaluation Dimension	Lambda Architecture Assessment	Kappa Architecture Assessment
Cost Efficiency	**Balanced Compute/Storage:** Hot stream processing uses fast RAM/CPU, while cold historical data resides in low-cost, commodity on-premises object storage (MinIO Parquet). Reduces expensive broker disk requirements.	**Storage Heavy:** Retaining high-volume streams long-term in Kafka topics for reprocessing creates huge cluster storage costs. Utilizing Tiered Storage (e.g., Kafka to S3) mitigates this but adds storage translation overhead.
Government Compliance	**100% Audit-Compliant:** Batch layer acts as an immutable, append-only source of truth. Weekly/ monthly ward reports generated from raw Parquet files guarantee audit reproducibility required for state legal submissions.	**Requires Strict Replay Guarantees:** Compliance reports generated from streaming windows must guarantee exact-once processing semantics across state resets. Any missed or duplicate stream event during replay violates audit integrity.

Architectural Choice & Final Justification

Selected Architecture: Hybrid Lambda-Inspired Kappa Platform ("Lambda-in-Practice")
For MetroConnect, we implement a **hybrid processing strategy**. We leverage **Apache Flink** as a pure Kappa-style stream processor for sub-second incident detection (AQI emergencies, traffic signal control, bus bunching) reading directly from Kafka. Concurrently, we utilize **Apache Spark Structured Streaming** reading from the same Kafka topics to write append-only Parquet files into on-premise object storage (MinIO). Spark reads these Parquet files for 15-minute ward aggregations and historical councillor reports.

Justification: This hybrid approach eliminates duplicate business logic where possible while honoring the strict government mandate: low-latency operational response is completely decoupled from heavy analytical batch workloads, guaranteeing that long-running councillor reports never degrade real-time signal control.

3. Government Smart City Architecture Readiness Checklist

This checklist establishes mandatory technical and operational governance criteria for deploying UrbanPulse within MetroConnect's municipal infrastructure, adhering strictly to Indian Government Smart City guidelines.

#	Domain / Category	Checklist Item & Technical Specification	Verification Method
1	**Data Sovereignty**	**Complete On-Premises Air-Gapped Hosting:** All data, Kafka brokers, processing nodes, and databases must reside exclusively within municipal government data centers (MetroConnect SDC). Zero external cloud dependencies or outbound internet routing permitted.	IP route audit & firewall isolation check
2	**Open-Source Mandate**	**100% FOSS Technology Stack:** Core infrastructure must exclusively utilize open-source software (Apache Kafka, Apache Flink, Apache Spark, PostgreSQL, Grafana). Zero proprietary vendor lock-in or commercial license dependencies allowed.	Software Bill of Materials (SBOM) audit
3	**Disaster Recovery (RPO)**	**Recovery Point Objective (RPO) < 15 Minutes:** Continuous offset commit checkpointing in Flink/Spark to state backends (MinIO/Ceph). Data loss in case of total site failure must not exceed 15 minutes of stream data.	Simulated broker kill & state restore test
4	**Disaster Recovery (RTO)**	**Recovery Time Objective (RTO) < 30 Minutes:** Automated failover mechanisms with Kubernetes (K8s) multi-zone node pools and standby Kafka brokers. Service restoration must occur within 30 minutes.	Active-Passive DR failover dry run
5	**Ward Officer UX**	**Non-Technical Ward Dashboard Accessibility:** Dashboard UI must feature localized language support (Marathi/Hindi/English), visual color-coded maps	User Acceptance Testing (UAT) with ward staff

#	Domain / Category	Checklist Item & Technical Specification	Verification Method
		(Red/Yellow/Green), and zero-SQL filter dropdowns tailored for non-technical elected ward officers.	
6	**Security & RBAC**	**Role-Based Access Control & TLS Encryption:** Mandatory mTLS encryption for all inter-broker and producer-consumer traffic. Strict RBAC restricting ward officers to their specific ward boundary data.	Penetration testing & access control audit
7	**Data Retention Policy**	**Tiered Lifecycle Management:** Enforce topic-level retention policies — Bus GPS (24 hours), Air Quality (90 days in Kafka, permanent in Parquet), Smart Meters (365 days) to comply with state energy audit laws.	Automated retention cron policy audit
8	**Fault Mitigation (DLQ)**	**Dead-Letter Queue Isolation:** Automatic routing of malformed, null-value, or out-of-range sensor payloads to `urbanpulse.dlq` topic with explicit error headers to prevent stream pipeline crashes.	Poison-pill payload injection test
9	**Interoperability**	**Open API Standards (NGSI-LD / REST):** Public-facing advisory APIs must adhere to NGSI-LD smart city data standards for seamless integration with emergency response services (112) and bus mobile apps.	OpenAPI schema validator compliance
10	**Observability**	**End-to-End Metrics & Lag Alerting:** Real-time Prometheus metrics monitoring consumer group lag, CPU utilization, and backpressure. Automated SMS/PagerAlerts if consumer lag exceeds 5,000 messages.	Prometheus alert manager simulation
11	**High Availability**	**Kafka Broker Resilience:** Minimum 3-broker cluster with topic replication factor of 3 (`min.insync.replicas=2`). Guarantees zero data loss even during single-broker hardware failure.	Node shutdown Chaos Engineering test
12	**Audit Logging**	**Immutable Administrative Audit Trail:** Every signal override, manual AQI threshold change, or system configuration update must be recorded in an append-only audit log database for legal scrutiny.	Immutable log entry verification check